Indian Institute of Technology (BHU), Varanasi

Optimising Fed-Batch Bioprocesses using pyFOOMB

UG Project Progress Report

Student: Yashashwi Singhania (23014019)
Guide: Dr. Sharon Mano Pappu J.
Department: Biochemical Engineering
Lab: BPDD (_____)

Project Overview

Objective

Optimise the **fed-batch feeding strategy** for recombinant protein production by exploiting the relationship between specific growth rate (μ) and specific product formation rate (q_p).

This Semester — Completed:

- ✓ Studied the pyFOOMB framework
- ✓ Built a full web-based GUI
- ✓ Surveyed μ–q_p models from literature
- ✓ Simulated batch X, S, P profiles

Next Steps:

- Implement fed-batch models in pyFOOMB
- Parameter estimation & feed optimisation
- Validate with experimental data
- Deploy GUI
- Scale-up analysis

What is pyFOOMB?

Python **F**ramework for **O**bject-**O**riented **M**odelling of **B**ioprocesses

Hemmerich et al. (2021), *Eng. Life Sci.* 21(3-4):242–257

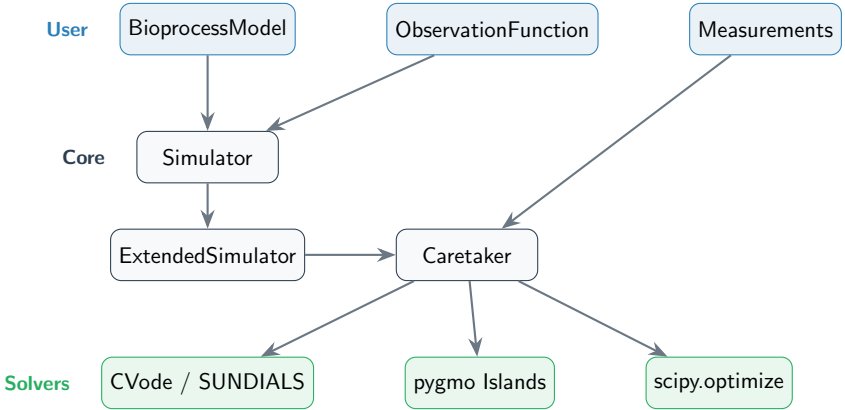
Core Capabilities:

- ODE-based bioprocess modelling
- User-defined kinetics in `rhs()` method
- Event handling for fed-batch (feed start/stop, bolus)
- Forward simulation via CVode (SUNDIALS)
- Parameter estimation: local (scipy) & global (pygmo)
- Sensitivity analysis, FIM, OED

Key Difference:

pyFOOMB does **not** hardcode kinetic models. Users implement *any* μ–q_p relationship in their model's `rhs()` method. The framework handles simulation, events, and estimation.

pyFOOMB Architecture



Users define ODE models → pyFOOMB handles integration, estimation & analysis

Our Contribution: pyFOOMB Web Platform

What already existed (pyFOOMB library):

- ODE modelling framework
- CVode integration (SUNDIALS)
- Parameter estimation engine
- Sensitivity & FIM analysis
- *Code-only* (Jupyter notebooks)

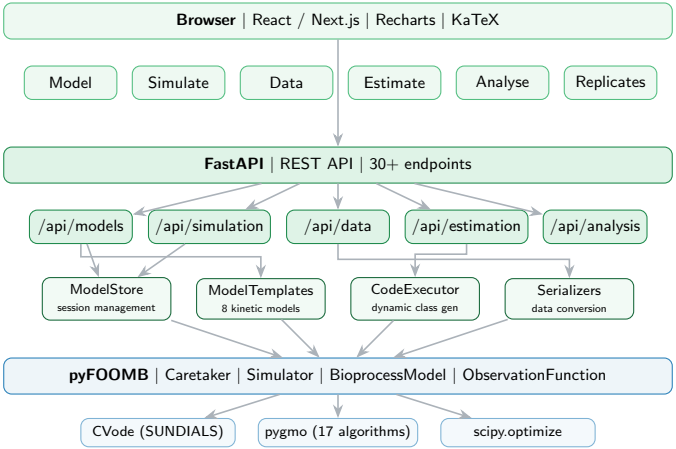
Problem: No visual interface. Requires Python programming to use any feature.

What we built (new this semester):

- ✓ Next.js web frontend (6 pages)
- ✓ FastAPI REST backend (30+ endpoints)
- ✓ 8 pre-built model templates
- ✓ Interactive plots & math rendering
- ✓ Data import (CSV, paste, Google Sheets)
- ✓ 39 automated API tests

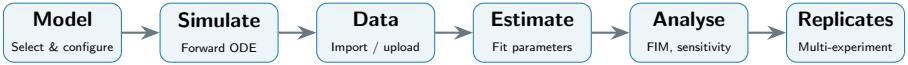
Result: Complete browser-based GUI wrapping 100% of the pyFOOMB API.

System Architecture



GUI: Modelling Workflow

The GUI provides a step-by-step pipeline from model definition to analysis:



Model Templates (8):

- Monod, Logistic, Exponential Growth
- Substrate Inhibition (Andrews)
- Contois, Double Monod
- Exponential Decay
- Fed-Batch Monod (with events)

Estimation Methods (5):

- Local (scipy L-BFGS-B, Nelder-Mead)
- Global parallel (pygmo, 17 algorithms)
- Repeated local from random starts
- Monte Carlo sampling
- Parallelised Monte Carlo

GUI: Interface

[Place your GUI screenshot here]

Save as `gui_screenshot.png` and uncomment the line above.

μ–q_p Relationship Models

The μ–q_p relationship governs how fast a cell produces product at a given growth rate. It determines the **optimal feeding strategy** in fed-batch.

Linear

$$q_p = \alpha\mu + \beta$$

Growth-associated.
Feed to maximise μ.

Bell-Shaped

$$q_p = q_p^{max} e^{-\frac{(\mu-\mu_{opt})^2}{2\sigma^2}}$$

Peak at μ_{opt}.
Maintain $\mu = \mu_{opt}$.

Hyperbolic

$$q_p = \frac{q_p^{max}\mu}{K_q+\mu}$$

Saturates at high μ.
Flexible feeding.

We surveyed 9 products from literature across *S. cerevisiae* and *P. pastoris*, and fitted each to one of these three models using `scipy.optimize.curve_fit`.

Literature Survey: 9 Products, 2 Organisms

μ–q_p data extracted from chemostat (CSTR) steady-state experiments at various dilution rates.

Product	**Organism**	**Model**	**Reference**	n
Resveratrol	*S. cerevisiae*	Linear	Vos et al. (2015)	5
VHH Antibody Fragments	*S. cerevisiae*	Linear	Frenken et al. (1998)	6
Crl1 Lipase (Single-Copy)	*P. pastoris*	Linear	Garrigós-Mart. (2019)	7
EPG Polygalacturonase	*S. cerevisiae*	Bell	Glauche et al. (2017)	10
Fab Antibody Fragment	*P. pastoris*	Bell	Rebnegger et al. (2014)	9
ROL Lipase	*P. pastoris*	Bell	Gasset et al. (2012)	8
Crl1 Lipase (Multi-Copy)	*P. pastoris*	Hyperbolic	Garrigós-Mart. (2019)	7
α-Galactosidase	*S. cerevisiae*	Hyperbolic	Hensing et al. (1998)	8
GFP	*P. pastoris*	Hyperbolic	Mattanovich et al. (2021)	8

n = number of steady-state data points per product. Parameters fitted using `scipy.optimize.curve_fit`.

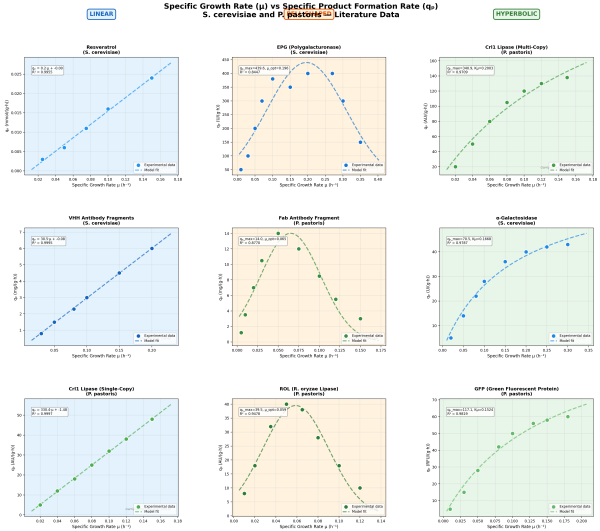
Extracted Data: μ and q_p Ranges

Data points digitised from each publication:

Product	**Model**	μ **range** ($\mathbf{h}^{-1}$)	q_p **range**	**Unit**	R^2
Resveratrol	Linear	0.025–0.15	0.003–0.024	mmol/(g·h)	0.99
VHH Antibody	Linear	0.03–0.20	0.8–6.0	mg/(g·h)	0.99
Crl1 Lipase (SCC)	Linear	0.02–0.15	5.0–48.0	AU/(g·h)	0.99
EPG	Bell	0.01–0.35	50–400	U/(g·h)	0.84
Fab Fragment	Bell	0.005–0.15	1.2–14.0	mg/(g·h)	0.96
ROL Lipase	Bell	0.01–0.12	8.0–40.0	AU/(g·h)	0.92
Crl1 Lipase (MCC)	Hyperbolic	0.02–0.15	20–138	AU/(g·h)	0.99
α-Galactosidase	Hyperbolic	0.02–0.30	5.0–43.0	U/(g·h)	0.99
GFP	Hyperbolic	0.01–0.18	5.0–60.0	RFU/(g·h)	0.99

All data obtained from continuous cultures where dilution rate $D = \mu$ at steady state. Fitted parameters: α, β (linear); $q_p^{max}, \mu_{opt}, \sigma$ (bell); q_p^{max}, K_q (hyperbolic).

μ–q_p Fitting Results



Batch X, S, P Profile Simulation

From the fitted μ–q_p parameters, we simulated full batch dynamics for each product.

ODE System (Monod batch):

$$\frac{dX}{dt} = \mu \cdot X$$

$$\frac{dS}{dt} = -\frac{\mu}{Y_{X/S}} \cdot X$$

$$\frac{dP}{dt} = q_p(\mu) \cdot X$$

where $\mu = \mu_{max} \cdot \dfrac{S}{K_S + S}$

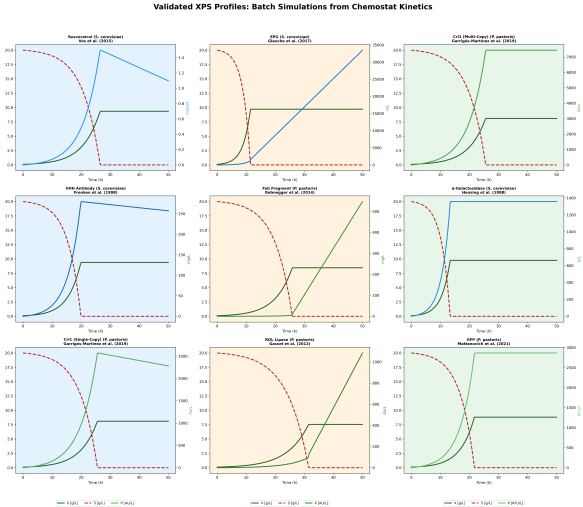
Integrated using `scipy.integrate.solve_ivp` (RK45).

Organism-Specific Parameters:

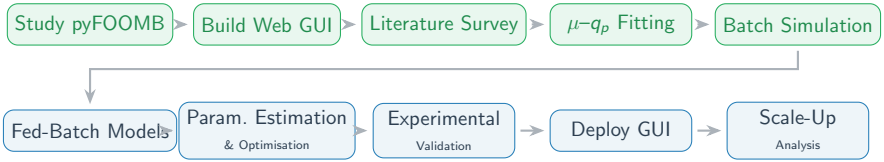
	S.c.	*P.p.*
μ_{max} (h^{-1})	0.40	0.20
K_S (g/L)	0.10	0.05
$Y_{X/S}$ (g/g)	0.50	0.45

$X_0 = 0.1$ g/L, $S_0 = 20$ g/L, $P_0 = 0$

XPS Simulation Results



Project Roadmap



Completed this semester:

- ✓ Full pyFOOMB web GUI
- ✓ 9-product μ–q_p literature survey
- ✓ Batch XPS profile simulations

Planned next:

- Fed-batch ODE models with feed events
- Fit & optimise feed profiles
- Validate against experimental data
- Deploy GUI

Thank You

Questions?

Student: Yashashwi Singhania (23014019)
Guide: Dr. Sharon Mano Pappu J.
Lab: BPDD, Dept. of Biochemical Engineering
Institute: IIT (BHU) Varanasi

April 2026